

## Leçon : Programmation

### Définition

Le **programme** explique comment l'ordinateur doit fonctionner. **C'est une suite d'instructions écrites par un humain dans un langage de programmation donné.** Elles sont destinées à indiquer à un ordinateur les actions à exécuter pour accomplir une tâche ou résoudre un problème. C'est le processeur qui exécute les instructions traduites au préalable en langage machine (0 et 1).

Exemples de langages de programmation : Python, langage C, Java, PHP, Javascript, CSS, HTML, assembleur...

### Quelques dates historiques

1949 : apparition du langage assembleur, 1<sup>ère</sup> représentation textuelle du langage machine lisible par l'humain

1964 : création du langage BASIC

1972 : invention du langage C

1991 : création du langage Python

### Variables et affectation

Les données utilisées par un programme sont stockées dans des variables. **Une variable est un espace mémoire dans lequel un programme stocke une valeur pour la mémoriser.** Ceci se fait par une **affectation** qui associe une donnée (représentée par une valeur ou une expression) avec un nom. L'opérateur d'affectation est noté " = ".

L'instruction `x = 3` associe la valeur 3 au nom x.

L'instruction `y = 3 + 5` associe la valeur de l'expression à droite du signe = (ici 8) au nom y.

**Règles pour définir le nom d'une variable : le nom d'une variable doit être explicite.** Il ne peut pas commencer par un nombre, ne doit pas contenir d'espace, ne doit pas être un mot réservé (if, return, True, False, def, else, not, and, or...) et ne doit pas contenir des caractères accentués. Le nom d'une variable peut contenir des lettres, des chiffres et le tiret du bas (underscore `_`). Exemple : `compte_epargne`, `note_QCM1`...

### Le type d'une variable définit l'ensemble des valeurs qui peuvent lui être affectées

Les types simples sont les types numériques pour représenter les nombres entiers et réels (`int` et `float`) et le type `str` (qui représente les chaînes de caractère). Le type `bool` (c'est-à-dire le type booléen) représente les valeurs True et False correspondant à Vrai ou Faux (ou encore à l'entier 1 pour True et l'entier 0 pour False).

NB : pour le type `float`, le séparateur entre la partie entière et la partie décimale est un point (Exemple : 3.14).

Il existe des types complexes appelés types structurés ou construits (tuples, tableaux, listes, dictionnaires).

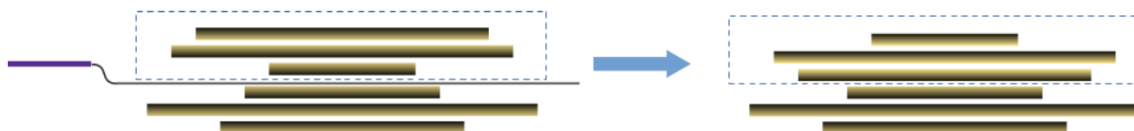
## ACTIVITÉ : Le crêpier psycho-rigide

### Situation

À la fin de sa journée, un crêpier dispose d'une pile de crêpes désordonnée. Le crêpier étant un peu psycho-rigide, il décide de ranger sa pile de crêpes, de la plus grande (en bas) à la plus petite (en haut), avec le côté brûlé caché.



Pour cette tâche, le crêpier peut faire une seule action : glisser sa spatule entre deux crêpes et retourner le haut de la pile.



### Question

Comment doit-il procéder pour trier toute la pile ?

- 1) Avec le matériel fourni, trouver comment faire par groupe de 2 ou 3.
- 2) Expliquer comment faire en langage naturel (écrire l'algorithme).
- 3) vérifier que votre solution est bien écrite avec un programmeur qui dicte ce qu'il faut faire et un processeur qui exécute.

L'algorithme du crêpier psychorigide montre l'importance de rédiger un algorithme avant de le traduire en langage de programmation en décomposant le problème en tâches simples (retourner le tas de crêpes, permuter les places) et en expliquant l'enchaînement de celles-ci.

De manière informelle on considère un algorithme comme un processus systématique pour réaliser un objectif (un gâteau, une division posée, une procédure de conversion d'un nombre en base 10 à la base 2).

## Définitions

Algorithme : suite d'opérations permettant de résoudre un type de problème en un nombre fini d'étapes.

Pseudo-code : Le pseudo-code permet d'écrire un algorithme en langage naturel, tout en introduisant un certain formalisme mais sans faire référence à un langage de programmation. Il n'existe pas de convention universelle pour l'écriture en pseudo-code mais nous utiliserons celles décrites ci-après.

### Structure générale du pseudo-code :

- Les déclarations décrivent les variables utilisées (en précisant le nom et le type : entier, réel...).
- Les instructions (une par ligne) sont indentées (décalées et alignées) pour délimiter les blocs d'instructions.
- L'algorithme est écrit entre les mots-clés Début et Fin.

#### EXEMPLE :

```
Déclarations
...
Début
...
Fin
```

### Déclaration et affectation de variables en pseudo-code :

- Les **variables** sont des symboles qui associent un nom à une valeur.
- Une **déclaration** est une instruction qui définit une variable par son identifiant et son type de donnée.
- L'affectation est l'action d'attribuer une valeur à une variable. Sa syntaxe est : **<variable> ← <valeur>**

L'affectation a lieu à l'intérieur d'un algorithme entre Début et Fin.

A une **variable** (age), on peut affecter :

- une valeur littérale de même type
- la valeur d'une expression de même type
- la valeur d'une autre variable de même type

#### EXEMPLE :

```
Déclarations
  age : Entier,
  ageDeRomain : Entier
Début
  age ← 12
  age ← age + 1
  ageDeRomain ← age
Fin
```

### Instruction conditionnelle en pseudo-code :

- L'instruction **si alors** permet de conditionner l'exécution d'un bloc d'instruction au résultat d'un test dont la valeur est *vrai* ou *faux* (on parle d'**expression booléenne**).

```
si <expression booléenne> alors
  <instructions si vrai>
fin si
```

- L'instruction **si alors sinon** permet aussi d'exécuter un bloc d'instructions si une condition est fausse.

```
si <expression booléenne> alors
  <instructions si vrai>
sinon
  <instructions si faux>
fin si
```

#### EXEMPLE :

```
si age < 0 alors
  age ← age + 1
sinon
  age ← age - 1
fin si
```

Itérations (répétitions) en pseudo-code : une itération est une suite d'instructions dont l'exécution s'effectue plusieurs fois. → On parle aussi de **boucle (bornée ou non bornée)**

- La **condition d'arrêt de la boucle** indique quand l'itération doit recommencer une exécution ou quitter la boucle.

**Boucle bornée** : le nombre d'itérations est connu et défini au début de la boucle. → instruction pour (**for**)

**Boucle non bornée** : on ne connaît pas à l'avance le nombre d'itération. → boucle tant que (**while**)

**Boucle bornée (for) :** l'instruction Pour permet d'exécuter des instructions en fonction d'une variation de valeur prédéfinie.

Sa syntaxe est :

```
pour <variable> ← <valeur début> à <valeur fin> faire  
    <bloc d'instructions>  
fin pour
```

Que fait cet algorithme ?

```
i : 0 somme 0  
i : 1 somme 1  
i : 2 somme 3  
i : 3 somme 6  
i : 4 somme 10  
i : 5 somme 15  
i : 6 somme 21  
i : 7 somme 28  
i : 8 somme 36  
i : 9 somme 45  
somme au total 45
```

EXEMPLE :

```
Déclarations  
    i, n, somme : Entier  
Début  
    n ← 10  
    somme ← 0  
    pour i ← 1 à n faire  
        somme ← somme + i  
    fin pour  
Fin
```

**Boucle non bornée (while) :** l'exécution de la prochaine boucle est conditionnée par une expression booléenne (cf. leçon sur l'architecture).

Sa syntaxe est :

```
tant que <expression booléenne> faire  
    <bloc d'instructions>  
fin tant que
```

Que fait cet algorithme ?

```
b : 5    q : 1    a : 15  
b : 5    q : 2    a : 10  
b : 5    q : 3    a : 5  
b : 5    q : 4    a : 0  
résultat final :  
a : 0  
b : 5  
q : 4
```

EXEMPLE :

```
Déclarations  
    a, b, q : Entier  
Début  
    a ← 20  
    b ← 5  
    q ← 0  
    tant que a > 0 faire  
        a ← a - b  
        q ← q + 1  
    fin tant que  
Fin
```

**Lecture/écriture en pseudo-code :** pour signifier l'interaction avec un utilisateur, il faut parfois introduire des actions de lecture par l'algorithme d'une valeur donnée par l'utilisateur.

Sa syntaxe est :

```
<variable> ← Lire()
```

Pour afficher un résultat à l'écran, on utilise la syntaxe suivante :

```
Écrire(<valeur à afficher>)
```

Que fait cet algorithme ? Exemples :

```
Quel âge as-tu ?19  
Tu es majeur
```

```
Quel âge as-tu ?17  
Tu es mineur
```

EXEMPLE :

```
Déclarations  
    age, attente : Entiers  
Début  
    Écrire("Quel est ton âge ? ")  
    age ← Lire()  
    si age ≥ 18 alors  
        Écrire("Tu es majeur.")  
    sinon  
        Écrire("Tu es mineur.")  
    fin si  
Fin
```

## Leçon : Programmation (fonctions, instructions conditionnelles, boucles)

**Les fonctions** : une **fonction** est un ensemble d'instructions qui peut recevoir un ou plusieurs arguments (valeurs ou variables) et qui peut renvoyer une ou plusieurs valeurs de retour. Structurer un programme avec des fonctions permet de les réutiliser sans avoir à réécrire les mêmes instructions. Elles organisent aussi le code et rendent le programme plus lisible. Elles permettent ainsi de construire des codes modulaires, plus faciles à lire, à réutiliser et à modifier.

Tout d'abord, **en Python une fonction se crée avec le mot-clé def**. Il faut ensuite écrire le nom de la fonction (mêmes règles que le nom des variables pour le choix du nom). Entre parenthèses, il faut écrire les paramètres séparés par des virgules. S'il n'y en a pas, laisser les parenthèses vides nom\_fonction(). **Terminer la ligne par deux points :**

Dans le corps de la fonction, il faut obligatoirement **indenter** les instructions en Python. L'indentation est le décalage à droite et l'alignement en début de ligne des instructions qui font partie d'un même bloc d'instructions.

Le bloc d'instructions peut se terminer par le mot clé **return** qui provoque la sortie immédiate de la fonction en renvoyant le contenu d'une ou plusieurs variables (séparées par des virgules s'il y en a plusieurs).

Enfin, une fonction doit être appelée pour être exécutée.

Corps de la fonction

```
def nom_fonction(parametre1, parametre2, etc.):  
    instruction1  
    instruction2  
    ...  
    return valeur_renvoyee_1, valeur_renvoyee_2, etc
```

**NB** : Dans l'appel de la fonction, il faut respecter l'ordre des paramètres s'il y en a plusieurs.

Variable locale / variable globale

```
def jeu():  
    for k in range(nb_repetitions):  
        print("hola")  
        valeur = 50  
        score = score + 1  
nb_repetitions = 5  
score = 0  
jeu()  
print(valeur)
```

lecture d'une variable définie dans le corps du programme : **autorisé**

modification d'une variable définie dans le corps du programme : **interdit**

lecture d'une variable définie dans le corps d'une fonction : **interdit**

Eviter d'utiliser des variables globales (définies dans le corps du programme en dehors des fonctions), utiliser des variables locales (définies à l'intérieur des fonctions) en cas de modification au sein d'une fonction.

Exemple :

```
def prix_a_afficher(quantite):  
    prix = 1.15 * quantite  
    return prix
```

Appel de la fonction et récupération de la valeur retournée :

quantite = 2  
prix\_a\_payer = prix\_a\_afficher(quantite)  
ou  
prix\_a\_payer = prix\_a\_afficher(2)

## Les instructions conditionnelles

L'instruction conditionnelle **if** permet d'exécuter des instructions en fonction d'une condition. Cette condition est une expression booléenne qui a pour valeur True ou False.

**Rappel** : le caractère **#** permet d'écrire des commentaires qui sont ignorés par l'interpréteur Python. Ils permettent au programmeur d'ajouter des explications sur le code ou de désactiver le code pendant le débogage sans le supprimer.

**Syntaxe générale :**

```
if expression booléenne :  
    # instructions à exécuter si l'expression booléenne est vraie  
else :  
    # instructions à exécuter si l'expression est fausse
```

**Syntaxe avec elif :**

```
if expression booléenne :  
    # instructions à exécuter si l'expression booléenne est vraie  
elif expression booléenne :  
    # instructions à exécuter si l'expression précédente est fausse et si celle-ci est vraie  
# elif expression booléenne :  
    # on peut mettre plusieurs elif s'il y a plusieurs cas  
else :  
    # instructions à exécuter si les expressions précédentes sont fausses
```

## Listes des opérateurs utilisés dans les expressions booléennes :

| Opérateur | Signification    | Opérateur | Signification       |
|-----------|------------------|-----------|---------------------|
| ==        | est égal à       | <=        | inférieur ou égal à |
| !=        | est différent de | >=        | supérieur ou égal à |
| <         | inférieur à      | in        | appartient à        |
| >         | supérieur à      | not in    | n'appartient pas à  |

On peut aussi utiliser les opérateurs **and**, **or**, **not** (cf. leçon au programme NSI en première)

## Exemple d'affichage en fonction de la moyenne obtenue : attention à l'ordre des cas et à l'indentation

```
1 moyenne = 7  
2  
3 if moyenne < 8:  
4     print("raté")  
5 elif moyenne < 10:  
6     print("repêchage")  
7 elif moyenne < 12:  
8     print("admis")  
9 elif moyenne < 14:  
10    print("mention AB")  
11 elif moyenne < 16:  
12    print("mention B")  
13 else:  
14    print("mention TB")
```

Les boucles permettent de répéter des blocs d'instructions. Il faut aussi respecter l'indentation au sein des boucles.

### Les boucles bornées (for)

Elles permettent d'exécuter un bloc d'instructions un nombre prédéfini de fois.

Syntaxe :

```
for i in range(n):  
    bloc_instructions
```

→ Ne pas oublier les : et l'indentation

→ Pour i allant de 0 à (n-1), exécuter n fois les instructions indentées d'un bloc.

→ i est incrémenté de 1 à chaque passage dans la boucle.

Exemples : l'indentation dans les deux exemples ci-dessous change le résultat de l'affichage.

```
for i in range(3):  
    print("Bonjour")  
print("Au revoir")
```

Bonjour  
Bonjour  
Bonjour  
Au revoir

```
for i in range(3):  
    print("Bonjour")  
    print("Au revoir")
```

Bonjour  
Au revoir  
Bonjour  
Au revoir  
Bonjour  
Au revoir

### Les boucles non bornées (while)

Elles permettent d'exécuter un bloc d'instructions tant qu'une expression booléenne est vraie. Elles s'arrêtent dès que la condition est fausse. La condition est une expression booléenne évaluée à True ou False.

Les boucles non bornées sont généralement utilisées lorsque le nombre d'itérations (ie. le nombre de passage dans la boucle) n'est pas connu à l'avance.

Syntaxe :

```
while condition :  
    bloc_instructions
```

Exemple :

```
i = 0  
while i < 3 :  
    print(i)  
    i = i + 1
```

0  
1  
2

Attention, avec **while**, il y a un risque de boucle infinie.

Documenter une fonction en utilisant la documentation (**docstring**) générée avec **help(nom\_fonction)**.

Exemple 1 : `help(print)` Exemple 2 : mettre les commentaires sous le nom de la fonction entre des triples guillemets.

```
def prix_articles(quantite, prix_unité):  
    """  
    calcule le prix total à payer en multipliant le nombre d'articles par son prix unitaire  
  
    paramètres :  
    -----  
    quantite (int) : nombre d'articles achetés  
    prix_unit (float): prix d'un seul article  
  
    retourne :  
    -----  
    rien  
  
    """  
    prix_total_article = prix_unité * quantite  
    print("Prix total (", quantite, " article(s)) :", prix_total_article, " euros.")  
  
prix_articles(2, 3)  
help(prix_articles)
```

Prix total ( 2 article(s)) : 6 euros.

Help on function prix\_articles in module \_\_main\_\_:

```
prix_articles(quantite, prix_unité)  
    calcule le prix total à payer en multipliant le nombre d'articles par son prix unitaire  
  
    paramètres :  
    -----  
    quantite (int) : nombre d'articles achetés  
    prix_unit (float): prix d'un seul article  
  
    retourne :  
    -----  
    rien
```

## Exercices (programmation)

**Exercice 1 :** Ecris l'appel de la fonction suivante pour effectuer les calculs :

a) 10/5

b) 3/2

c) 1/3

```
def division(x,y):  
    """  
    Retourne la division de x par y si y différent de 0  
    """  
    # vérifier si y est bien différent de zéro  
    if (y != 0) :  
        print("Division ", x, " / ", y, " = ",x/y)  
        return x/y  
    else :  
        print("Division par zéro impossible")  
        return None
```

**Exercice 2 :** La fonction **repete** prend en paramètres une chaîne de caractère **mot** et un entier **x**. Complète cette fonction dont l'appel `repete("hello",3)` produit l'affichage suivant :

```
def repete(mot,x):  
    for .....  
        print(mot)  
  
repete("hello",3)
```

```
hello  
hello  
hello
```

**Exercice 3 :** Ecris une fonction **double(x)** qui renvoie le double de la valeur passée en paramètre si cette valeur est positive (ie. supérieure ou égale à 0). Si cette valeur est négative, la fonction renvoie un message.

**Exercice 4 :** Que renvoie l'appel `quelRenvoi(3,2)` de la fonction ci-contre ?

```
def quelRenvoi(a,b):  
    c = a + 1  
    d = b**2  
    return c - d  
  
quelRenvoi(3,2)
```

**Exercice 5 :** On considère le code ci-contre. Réécris ce code en utilisant une boucle **while**.

```
for i in range(5):  
    print("hello")  
  
hello  
hello  
hello  
hello  
hello
```

**Exercice 6 :** Ecrire un programme qui indique si une année est bissextile. Une année est bissextile si elle est divisible par 4. Cependant, les siècles ne sont pas bissextiles, sauf les multiples de 400.

**Exercice 7 :** Ecris l'algorithme pour afficher les n premiers nombres.

**Exercice 8 :** Ecris l'algorithme du PGCD.

**Exercice 9 :** a) Quelle est la valeur de **a** à la fin de l'exécution du programme ci-contre ?

b) Qu'affiche ce programme ?

```
a = 21  
  
def double(x)  
    print(x * 2)  
  
a = double(a)
```